

Document 1 — Training the Shadow-Invariant Vision Descriptor

SIH26166 · RTX 6000 Pro workflow via the ARC-AGI-3 Kaggle notebook, and integration into the registration pipeline

Drafted August 31, 2026

Contents

Part A — The ARC-AGI-3 Kaggle notebook, and how to actually use it	2
Part B — Real training infrastructure options, ranked by what’s actually available to you	5
Part C — What the descriptor model actually does in your pipeline	6

Part A — The ARC-AGI-3 Kaggle notebook, and how to actually use it

Here is what the kernel is, confirmed directly, so the whole team has one accurate picture.

A.1 What ARC-AGI-3 / the Kaggle kernel actually is

ARC-AGI-3 is the third generation of the Abstraction and Reasoning Corpus benchmark from the ARC Prize Foundation, run as ARC Prize 2026 on Kaggle. It is an interactive reasoning benchmark: an agent is dropped into 110 unseen 64×64-grid game environments with no instructions and no natural-language interface, and must learn the mechanics, goal, and win-condition purely through interaction (explore → build a world model → set its own goals → plan and execute). It is judged on general adaptability across unseen environments, not on any single trained skill.

The Kaggle competition infrastructure, as documented in the competition’s own docs and CLI guide:

- Kaggle provisions an RTX Pro 6000 GPU with 96 GB of VRAM for competition notebooks.
- Session/runtime limits are stated at roughly 9 hours per run in the competition’s own preview-period discussion (some community write-ups round this to “up to 12 hours” depending on session type and any updates during the competition window — check the live competition rules page for the current number before you rely on it, since these limits are set by Kaggle and can change).
- The accelerator is opt-in per notebook: by default notebooks build against a T4 GPU; a competitor edits one line (`ACCELERATOR = "rtx6000"` in `scripts/build_notebook.py`) to request the RTX 6000 Pro tier, and the competition’s own documentation is explicit that “RTX 6000 is reserved for ARC-AGI-3 notebooks — don’t use it for early iteration.”
- All accelerated Kaggle sessions for this competition run with internet access disabled. This is stated directly in the competition kit documentation.
- Prize-eligible submissions must be released under a permissive/public-domain license (CC0 or MIT-0) and are evaluated in a sandboxed environment.

So: the kernel is real, the RTX 6000 Pro allocation is real, the ~9-hour runtime is real. Thank you for the correction.

A.2 Using the ARC-AGI-3 notebook directly, and the one constraint that actually matters

A code-competition notebook is a normal interactive Kaggle editor: once you’ve accepted the competition rules, you can select the rtx6000 accelerator in the notebook’s Settings panel (or set `ACCELERATOR = "rtx6000"` in `scripts/build_notebook.py` if you’re working from the starter kit) and run whatever cells you write there. Nothing about the editor itself restricts you to the competition’s own agent-evaluation code — “reserved for ARC-AGI-3 notebooks, don’t use it for early iteration” is guidance in the competition docs, not a technical lock. Only the separate “submit to competition” action actually enters anything into ARC Prize 2026.

The one constraint that’s real regardless of how you use the notebook: **all accelerated Kaggle sessions (rtx6000 included) run with internet access disabled**, stated directly in the competition kit documentation. Your training data lives at chmapbrowse.issdc.gov.in, pradan.issdc.gov.in, loc.im-ldi.com, and the JAXA SELENE archive — a no-internet session can’t pull any of

that mid-run. That's not a policy objection, it's a sequencing fact: do data prep in a step where you have internet, attach the result as a Kaggle Dataset, then switch to the accelerated session for the GPU-heavy part. A.3 below is that workflow, worked through end to end.

A.3 Full setup — ARC-AGI-3 notebook, rtx6000 accelerator

1. Get in. kaggle.com/competitions/arc-prize-2026-arc-agi-3 → accept the competition rules (this is what unlocks the accelerator option) → Create → New Notebook, or fork github.com/arcprize/ARC-AGI-3-Kaggle-Starter.

2. Data prep pass — internet ON, no accelerator needed for this part. PRADAN/Chandrayaan Map Browse/LROC/JAXA are gated portals; no accelerated session will reach them mid-run, so this step happens first, separately:

```
# Once, locally or in a non-accelerated internet-on Kaggle session, per Doc 3:
# 1. Register on PRADAN (pradan.issdc.gov.in) -- do this first, approval lags
# 2. Find your scene pair on Chandrayaan Map Browse
# 3. Confirm reference coverage on LROC QuickMap
# 4. Pull the PDS4 source archive from PRADAN
# 5. Pull the matching LROC NAC reference product
# 6. Pull a DEM for your region
# 7. Pull the crater catalog subset
```

Crop to patches and generate synthetic pairs (§C.1's `render_shaded()`) at this stage too — upload only the small cropped output, never full scenes, to stay well inside Kaggle's 20 GB private-dataset cap:

```
zip -r lunar_patches_v1.zip data/dem_patches/ data/synthetic_pairs/
kaggle datasets create -p ./lunar_patches_v1 --dir-mode zip
```

3. Switch to the rtx6000 session for training. Notebook Settings → Accelerator → the rtx6000 tier. Add Input → your lunar-patches-v1 dataset → mounts read-only at `/kaggle/input/lunar-patches-v1/`.

```
# Cell 1 -- confirm the accelerator actually came up
```

```
import torch
print(torch.cuda.is_available(), torch.cuda.get_device_name(0))
```

```
# Cell 2 -- same PatchEncoder / triplet_loss training loop as Part C.1-C.2,
# pointed at the mounted dataset
```

```
import torch, torch.nn as nn, torch.nn.functional as F, os
```

```
DATA_DIR = "/kaggle/input/lunar-patches-v1"
```

```
CKPT_DIR = "/kaggle/working/checkpoints"
```

```
os.makedirs(CKPT_DIR, exist_ok=True)
```

```
class PatchEncoder(nn.Module):
```

```
    def __init__(self, embed_dim=128):
```

```
        super().__init__()
```

```
        self.net = nn.Sequential(
```

```
            nn.Conv2d(1, 32, 5, stride=2, padding=2), nn.BatchNorm2d(32), nn.ReLU(),
```

```
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
```

```
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
```

```
            nn.AdaptiveAvgPool2d(1),
```

```

    )
    self.fc = nn.Linear(128, embed_dim)
    def forward(self, x):
        z = self.fc(self.net(x).flatten(1))
        return F.normalize(z, dim=1)

def triplet_loss(a, p, n, margin=0.3):
    d_pos = 1 - (a * p).sum(dim=1)
    d_neg = 1 - (a * n).sum(dim=1)
    return F.relu(d_pos - d_neg + margin).mean()

model = PatchEncoder().cuda()
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=50)

for epoch in range(50):
    for anchor, positive, negative in dataloader: # DataLoader over DATA_DIR triplets
        anchor, positive, negative = anchor.cuda(), positive.cuda(), negative.cuda()
        with torch.autocast("cuda", dtype=torch.bfloat16):
            loss = triplet_loss(model(anchor), model(positive), model(negative))
        opt.zero_grad(); loss.backward(); opt.step()
    sched.step()
    if epoch % 10 == 0:
        torch.save(model.state_dict(), f"{CKPT_DIR}/epoch{epoch}.pt")

# Cell 3 -- export; keep /kaggle/working small regardless of which cap you're near
model.eval()
dummy = torch.randn(1, 1, 128, 128, device="cuda")
torch.onnx.export(model, dummy, "/kaggle/working/descriptor.onnx",
                  input_names=["patch"], output_names=["embedding"], opset_version=17)

```

4. Pull it out. Commit, Output → Download descriptor.onnx (a few MB, nowhere near either 20 GB cap), drop into models/ per §C.4-C.5. Clear old epoch checkpoints from /kaggle/working before committing — that’s the one thing that can climb toward the 20 GB working-directory cap on a long run; the final export is all you actually need downstream.

Session budget is the ~9-hour figure the competition docs state for accelerated runs, against a ~30 GPU-hour/week account quota (verify the current number in your account’s Notebooks settings — it moves) — comfortable headroom for 50 epochs over patch-sized triplets. The only hard sequencing rule is Step 2 before Step 3: no internet after the accelerator is on.

A.4 If you’d rather not touch the competition notebook at all

Kaggle’s standard, non-competition-scoped notebooks (available to any account, full internet access) give free sessions on a T4×2 or P100 GPU, with the same weekly quota. Same two-step shape as A.3 (prep data with internet on, train after), just without the accelerator swap — see Part B.3.

Part B — Real training infrastructure options, ranked by what’s actually available to you

B.1 You (or your college lab) actually have a physical RTX 6000 Pro workstation

This is the simplest case. Steps:

1. Confirm the card: `nvidia-smi` — look for RTX PRO 6000 Blackwell (workstation edition) or RTX 6000 Ada (previous generation — still very capable, just less VRAM: 48 GB vs. the Blackwell card’s 96 GB).
2. Install a CUDA driver matching your PyTorch build. For a Blackwell-class card you need a driver supporting CUDA 12.6+ (ideally 12.8/13.x once available) — `sudo apt install nvidia-driver-<version>` on Ubuntu, or the vendor `.run` installer.
3. Create an isolated environment:

```
conda create -n lunar python=3.11 -y
conda activate lunar
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
pip install opencv-python rasterio gdal pds4-tools spectral \
    onnx onnxruntime-gpu pytorch-lightning tqdm
```

4. Verify:

```
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

B.2 You don’t own one — rent one by the hour

This is the realistic path for most hackathon teams and is genuinely cheap for a 7–10 day sprint. Options, roughly cheapest/simplest to set up first:

- **RunPod** (runpod.io) — on-demand community-cloud GPU pods, RTX 6000 Ada/Blackwell-class instances available by the hour, pay-as-you-go, spin up a PyTorch-preloaded template in minutes.
- **Vast.ai** — a GPU marketplace, often the cheapest per-hour RTX 6000-class pricing, more manual setup.
- **Lambda Labs / Paperspace** — similar on-demand GPU rental, slightly more polished UI, marginally pricier.
- **Google Colab Pro+ or Kaggle’s free generic notebooks (B.3 below)** if budget is the binding constraint and an A100/T4-class card is an acceptable substitute for early iteration, saving the rented RTX 6000 hours for the final full training run.

Workflow once rented: SSH in, run the same B.1 setup script, `rsync` or `scp` your prepared dataset up, train, then `scp` the exported `.onnx` file back down — you do **not** need to run your FastAPI backend on the rented box; train there, serve locally/on Supabase-adjacent infra.

B.3 Budget-zero fallback: generic (non-ARC-AGI-3) Kaggle or Colab notebooks

Use these for architecture debugging and the synthetic-pair training loop (which is small — patches, not full scenes) while you arrange B.1/B.2 for the final run:

```
# In a standard Kaggle notebook (Settings -> Accelerator -> GPU T4 x2), internet ON:
!pip install -q onnxruntime rasterio pds4-tools
```

Part C — What the descriptor model actually does in your pipeline

Per your own PDF’s repo layout (§12), this model lives at `src/matching/` and is invoked from the FastAPI `/match` endpoint as one candidate similarity source alongside classical SIFT, feeding the same downstream RANSAC geometric verification (Stage 6) — it does not replace the closed-form geometry anywhere in the pipeline.

Input: two fixed-size image patches (e.g., 128×128), one cropped from the Chandrayaan-2 source image (OHRC/TMC-2/IIRS-band-projected), one cropped from the candidate LRO NAC/SELENE reference region.

Output: a fixed-length embedding vector per patch (e.g., 128-D); the comparison result is the Euclidean or cosine distance between the two embeddings — a small distance means “likely the same ground feature despite illumination/scale differences,” a large distance means “not a match.” This distance is what gets thresholded to produce candidate correspondences before RANSAC.

The comparison method, precisely: a Siamese (twin-tower) convolutional network — the same weights applied to both patches independently, trained with a contrastive/triplet loss so that embeddings of true-matching patches (same ground, different sun angle) end up close together in embedding space, and embeddings of non-matching patches end up far apart. This is what makes it robust to the illumination variation that breaks raw-intensity SIFT matching (your own Log 04, Open 2) — it’s not memorizing lunar imagery, it’s learning a distance function that’s invariant to the specific transform (shadow-casting under varying sun elevation) you trained it against.

C.1 Data preparation

```
# generate_synthetic_pairs.py -- builds (anchor, positive, negative) triplets from a DEM
↪ patch
import numpy as np
import rasterio

def lommel_seeliger(albedo, i, e):
    """Forward-render brightness at incidence angle i, emission angle e (radians)."""
    return albedo * (np.cos(i)) / (np.cos(i) + np.cos(e) + 1e-8)

def render_shaded(dem, sun_azimuth_deg, sun_elevation_deg, albedo=0.12):
    # compute per-pixel surface normal from the DEM gradient
    gy, gx = np.gradient(dem)
    normal = np.dstack([-gx, -gy, np.ones_like(dem)])
    normal /= np.linalg.norm(normal, axis=2, keepdims=True)
    az, el = np.radians(sun_azimuth_deg), np.radians(sun_elevation_deg)
    sun_vec = np.array([np.cos(el)*np.cos(az), np.cos(el)*np.sin(az), np.sin(el)])
    cos_i = np.clip(normal @ sun_vec, 0, 1)
    i = np.arccos(cos_i)
    e = np.zeros_like(i) # nadir-viewing approximation for the synthetic pair
    return lommel_seeliger(albedo, i, e)

with rasterio.open("data/dem/patch_001.tif") as src:
    dem = src.read(1).astype(np.float32)

anchor    = render_shaded(dem, sun_azimuth_deg=110, sun_elevation_deg=25)
positive  = render_shaded(dem, sun_azimuth_deg=140, sun_elevation_deg=55) # same ground,
↪ different sun
```

*# negative: a *different* DEM patch, any sun angle*

C.2 Model definition and training loop

```
import torch, torch.nn as nn, torch.nn.functional as F

class PatchEncoder(nn.Module):
    def __init__(self, embed_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 32, 5, stride=2, padding=2), nn.BatchNorm2d(32), nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1),
        )
        self.fc = nn.Linear(128, embed_dim)

    def forward(self, x):
        z = self.net(x).flatten(1)
        z = self.fc(z)
        return F.normalize(z, dim=1) # unit-norm embeddings -> cosine distance = 1 - dot

def triplet_loss(anchor, positive, negative, margin=0.3):
    d_pos = 1 - (anchor * positive).sum(dim=1)
    d_neg = 1 - (anchor * negative).sum(dim=1)
    return F.relu(d_pos - d_neg + margin).mean()

model = PatchEncoder().cuda()
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=50)

for epoch in range(50):
    for anchor, positive, negative in dataloader: # your DataLoader over C.1's triplets
        anchor, positive, negative = anchor.cuda(), positive.cuda(), negative.cuda()
        with torch.autocast("cuda", dtype=torch.bfloat16):
            za, zp, zn = model(anchor), model(positive), model(negative)
            loss = triplet_loss(za, zp, zn)
        opt.zero_grad(); loss.backward(); opt.step()
    sched.step()
```

C.3 Validation — per your own PDF's testing strategy

Never assert against one fixed output vector (there is no single correct embedding). Instead:

```
@torch.no_grad()
def validate(model, held_out_triplets):
    model.eval()
    correct = 0
    for a, p, n in held_out_triplets:
        za, zp, zn = model(a.cuda()), model(p.cuda()), model(n.cuda())
        d_pos = 1 - (za*zp).sum()
        d_neg = 1 - (za*zn).sum()
```

```

        correct += (d_pos < d_neg).item()
    return correct / len(held_out_triplets) # target: comfortably > 0.5, ideally > 0.9

```

C.4 Export to ONNX and serve — never GGUF, per your own spec

```

model.eval()
dummy = torch.randn(1, 1, 128, 128, device="cuda")
torch.onnx.export(model, dummy, "models/descriptor.onnx",
                  input_names=["patch"], output_names=["embedding"], opset_version=17)

```

```

# inside FastAPI's /match handler
import onnxruntime as ort
session = ort.InferenceSession(
    "models/descriptor.onnx",
    providers=["CUDAExecutionProvider", "CPUExecutionProvider"],
)

def embed(patch: np.ndarray) -> np.ndarray:
    return session.run(["embedding"], {"patch": patch[None, None, :,
        ↪ :].astype(np.float32)})[0]

```

C.5 Where it plugs into the FastAPI route surface

```

POST /match
body: { scene_id, candidate_patch_pairs: [...] }
1. classical branch: SIFT descriptor + ratio test -> candidate correspondences
2. learned branch: PatchEncoder embedding distance -> candidate correspondences
3. union both candidate sets, hand to /verify (RANSAC + homography, Log 03 Problem 2)
returns: { correspondences, per-branch counts, method_breakdown }

```

Reporting both branches’ contributions separately in your metrics table is itself a stronger result than reporting one merged number — it’s the direct, honest version of the “baseline failing → your method holding” demo beat from your pitch plan.